Monitoreo de la API con New Relic

Buscador eficiente de documentos
Etapa 4: profiling avanzado y APM

Campaña de carga del 15 de octubre de 2026

Aplicación en New Relic: pef-26
Tecnologías: Python 3.14, FastAPI, Uvicorn y agente New Relic Python 14.0.0
Endpoints observados: GET /buscar y POST /reindexar
Corpus: 50.000 documentos

1. Resumen

La API del buscador se instrumentó con el agente de New Relic y se sometió a una campaña de carga de tres escenarios, generada con un script propio. Con los datos que envió el agente se armó un dashboard que muestra cuánto tarda cada búsqueda minuto a minuto, el consumo de RAM y la CPU del servidor, que es lo que pide la consigna.

Hallazgos principales

- **Sin errores.** El servidor atendió 81.979 solicitudes y todas respondieron 200, también durante las recargas del índice.
- **Con un usuario, la mitad de las búsquedas tarda menos de 1,6 ms dentro del servidor.** Con 20 usuarios simultáneos la mediana sube a 145 ms, pero el rendimiento no aumenta: se mantiene entre 107 y 117 búsquedas por segundo.
- **El servidor usa un solo núcleo.** La CPU del proceso se mantiene entre el 20 % y el 24 % de la máquina, y el 25 % equivale a un núcleo de los cuatro lógicos. Los hilos que atienden las búsquedas se turnan el GIL de Python, así que más usuarios solo agregan espera.
- **Cada recarga del índice tarda unos 16 s**, y el pico de RAM del proceso pasa de unos 230 MB a unos 590 MB, porque mientras se construye el índice nuevo el viejo sigue en memoria.
- **New Relic muestra qué es lento, pero no por qué.** El 98,9 % del tiempo de /reindexar queda dentro de una sola función, sin más detalle. Esa pregunta la responde el perfilado con cProfile, SnakeViz y Scalene, en la sección «Perfilado de la versión refactorizada» del README.

2. Cómo se instrumentó la API

La API se levanta con `src/servidor.py`. Si la variable `NEW_RELIC_LICENSE_KEY` está definida (se lee de un archivo `.env` que no se sube al repositorio), el lanzador inicializa el agente antes de importar FastAPI, que es lo que el agente necesita para instrumentarla. Sin licencia, la API arranca igual, sin monitoreo.

```
python src/servidor.py --env-file .env
```

El código del buscador no se modificó. El agente reconoce FastAPI y registra cada solicitud como una transacción con el nombre de su endpoint, por ejemplo `api:create_app.<locals>.buscar`. Sin configuración adicional envía las transacciones (duración, cantidad y errores), la CPU y la memoria del proceso, y los logs de Uvicorn.

No se instaló el agente de infraestructura de New Relic. Es un servicio del sistema que mide la computadora entera, incluidos el editor y el propio generador de carga. Las métricas del servidor de este informe vienen del agente de la aplicación y corresponden solo al proceso de la API.

Un problema encontrado en el camino. En la primera corrida los datos llegaron a New Relic con el nombre por defecto, «Python Application», aunque la consola decía pef-26. El agente lee `NEW_RELIC_APP_NAME` en el momento en que se importa, y el lanzador lo definía después del import. Se corrigió el orden, se agregó un test que lo verifica y se repitió la campaña completa: todos los datos de este informe son de la segunda corrida.

3. La campaña de carga

La carga la genera `src/generar_trafico.py`. Cada usuario simulado es una tarea asíncrona que envía una búsqueda, espera la respuesta y envía la siguiente, sin pausas. El script mide la latencia desde el cliente. La mezcla de consultas fue la misma en los tres escenarios:

- **60 % repetidas:** cinco consultas fijas, que después de la primera vez salen de la caché de la API.
- **30 % variadas:** de una a tres palabras al azar del vocabulario del corpus, con `top_k` entre 1 y 20. Casi nunca se repiten, así que se calculan desde cero.
- **10 % inexistentes:** palabras que no están en el índice.

Escenario	Usuarios simultáneos	Recarga del índice	Hora local (UTC-3)	Hora en New Relic (UTC)
1	1	No	15:51:13 a 15:55:14	18:51 a 18:55
2	20	No	15:56:44 a 16:00:45	18:56 a 19:00
3	20	Cada 60 s	16:02:15 a 16:06:16	19:02 a 19:06

Cada escenario duró 4 minutos, con 90 s de pausa entre uno y otro para que se distingan en los gráficos. New Relic muestra las horas en UTC, tres horas más que la hora local. El servidor y el generador corrieron en la misma computadora (Intel Core i5-7267U, con 2 núcleos físicos y 4 lógicos), con el índice de 50.000 documentos ya guardado en disco.

4. Lo que midió el cliente

Escenario	Búsquedas	Por segundo	Media	p50	p95	p99	Máximo	Errores
1	27.964	116,4	8,5 ms	3,0 ms	28,8 ms	36,3 ms	234,9 ms	0
2	25.832	107,5	185,8 ms	169,4 ms	350,9 ms	468,6 ms	672,0 ms	0
3	28.179	117,3	170,3 ms	155,6 ms	307,9 ms	481,9 ms	1.319,0 ms	0

En el escenario 1, la mediana de las consultas repetidas y de las inexistentes es de unos 3 ms, y la de las variadas, de 22 ms: ese es el costo real de una búsqueda sin caché. En el corpus sintético cada término aparece en alrededor del 95 % de los documentos, así que una búsqueda variada puntúa decenas de miles de candidatos.

En el escenario 3 hubo tres recargas del índice: tardaron 16,4 s de media y 19,0 s la más lenta. Mientras tanto la API siguió respondiendo con el índice anterior, sin errores.

5. Lo que muestra New Relic

Las capturas se tomaron con la interfaz en inglés y la hora en UTC. La sección 9 traduce los indicadores.

5.1 Resumen del servicio

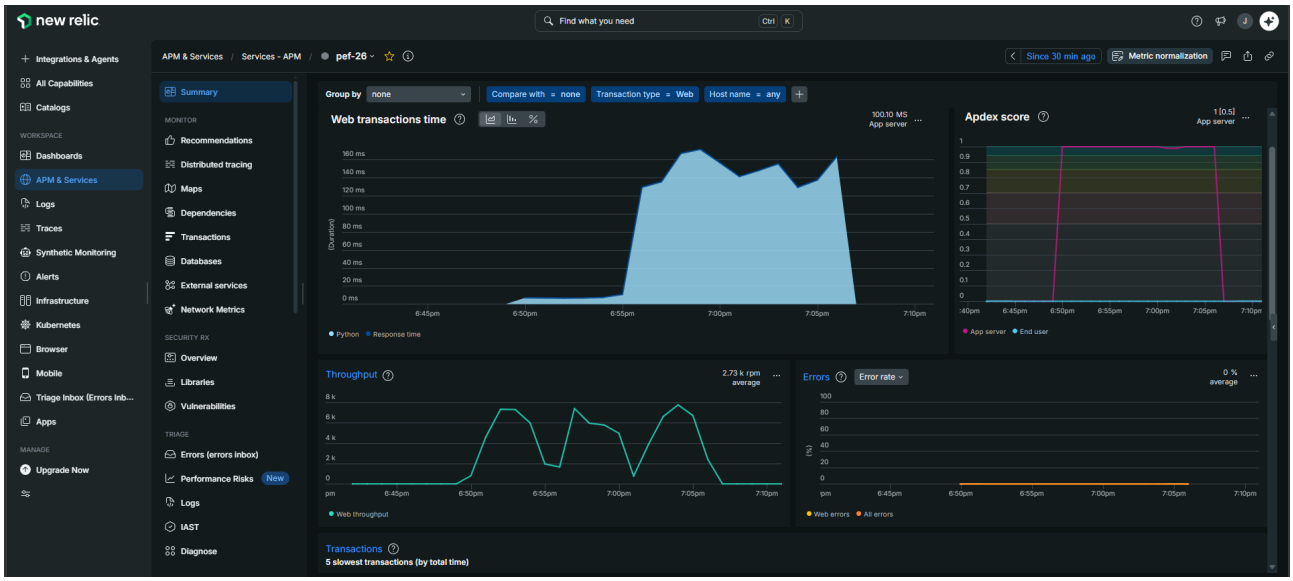


Figura 1. Resumen de pef-26: tiempo de respuesta, Apdex, rendimiento y errores durante la campaña.

Se distinguen los tres escenarios. El tiempo de las transacciones web se mantiene en pocos milisegundos con un usuario y sube a 120-160 ms con 20. El rendimiento llega a unas 7.000 solicitudes por minuto, unas 116 por segundo, lo mismo que midió el cliente. La tasa de errores es 0 % y el Apdex, con un umbral de 0,5 s, se mantiene en 1: casi todas las búsquedas responden en menos de medio segundo.

5.2 Transacciones

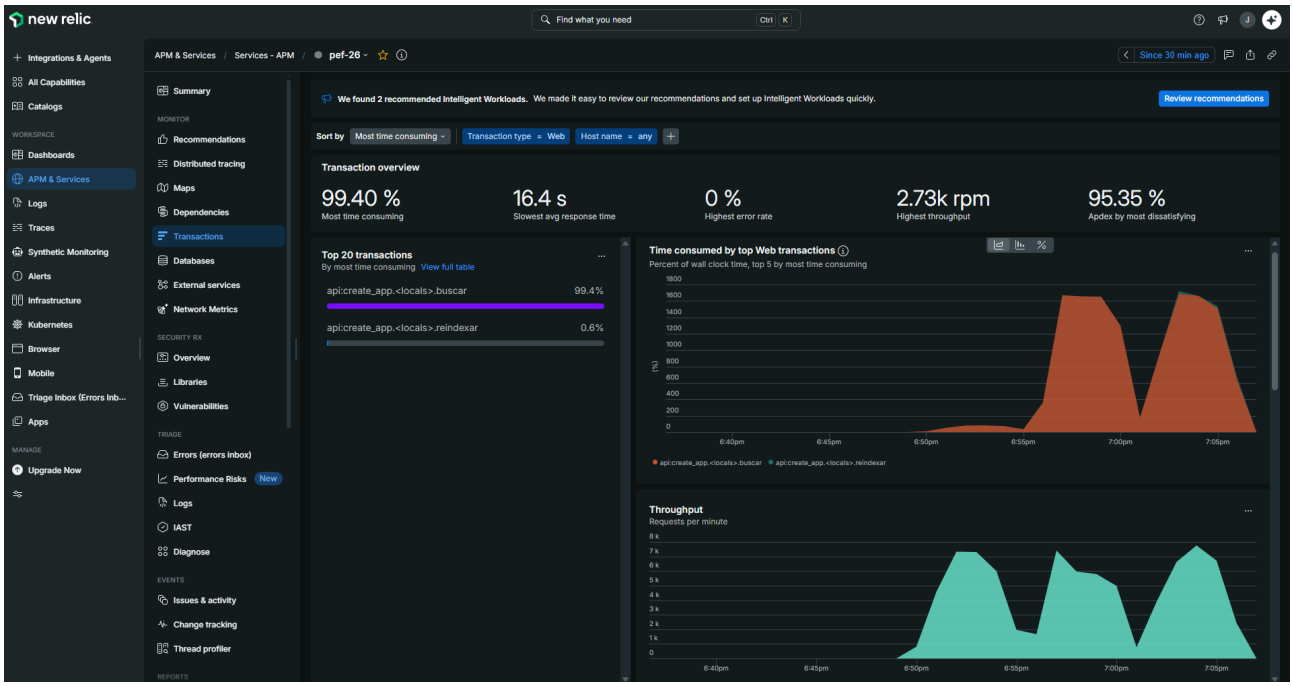


Figura 2. Transacciones ordenadas por tiempo consumido.

Hay dos transacciones. /buscar ocupa el 99,4 % del tiempo total del servidor por volumen: se llamó unas 82.000 veces. /reindexar ocupa el 0,6 %, pero es la más lenta, con 16,4 s de promedio.

El gráfico de tiempo consumido mide qué porcentaje del tiempo de reloj pasa el servidor dentro de cada transacción. En los escenarios 2 y 3 ronda el 1.600-1.800 %: en promedio hay entre 16 y 18 búsquedas

en curso dentro de la aplicación, de los 20 usuarios. El resto está en tránsito (la red, la cola de Uvicorn o el propio cliente), lo que coincide con la diferencia de latencia entre el servidor y el cliente de la sección 6.

5.3 Métricas del servidor: CPU y memoria



Figura 3. CPU y memoria del proceso de la API (recorte de la vista de transacciones).

La vista de transacciones también muestra la CPU y la memoria del proceso, que el agente de la aplicación informa sin el agente de infraestructura. La CPU se mantiene entre el 20 % y el 24 % durante la carga y cae casi a cero en las pausas. El porcentaje es sobre la máquina entera, con 4 CPU lógicas, así que el 25 % equivale a un núcleo completo: el proceso usa un núcleo y deja los otros tres casi libres, aun con 20 usuarios. Incluso con un solo usuario está en un 21 %, porque el cliente no hace pausas y el servidor casi nunca está ocioso.

La memoria se mantiene plana hasta el escenario 3 y ahí sube por escalones, uno por cada recarga del índice. Los valores en MB están en el dashboard (sección 5.5).

5.4 La recarga del índice

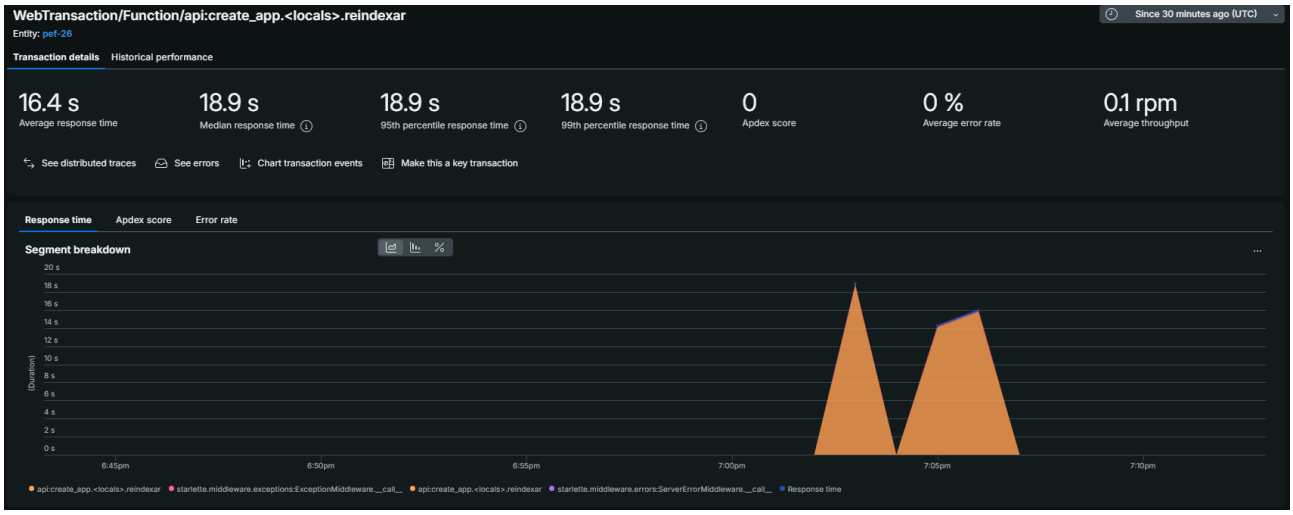


Figura 4. Detalle de la transacción /reindexar.

La recarga tarda 16,4 s de promedio, y la mediana, el p95 y el p99 dan 18,9 s (con tres recargas, los percentiles dicen poco). El cliente midió 16,4 s de media y 19,0 s la más lenta: coinciden. El Apdex es 0 porque todas las recargas superan los 2 s (cuatro veces el umbral), lo esperable para una operación que reconstruye el índice completo.

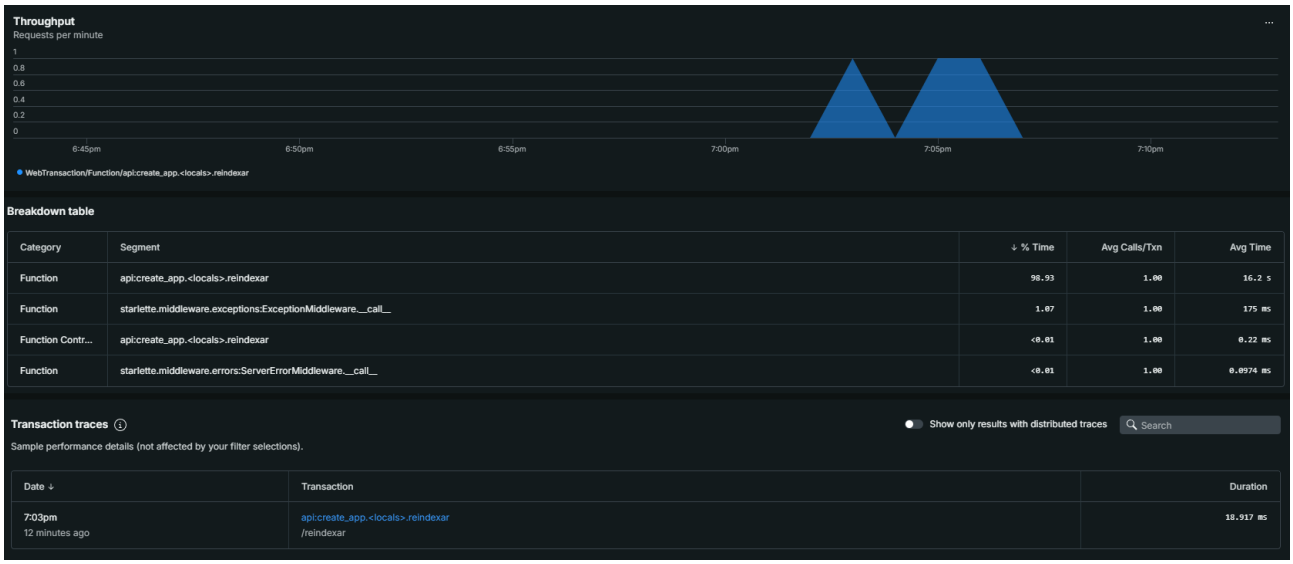


Figura 5. Desglose del tiempo de /reindexar y la traza más lenta.

La tabla de desglose es la parte más útil: el 98,9 % del tiempo (16,2 s por llamada) queda dentro de la función `reindexar`, sin más detalle. El resto, unos 175 ms, queda en el middleware de Starlette que envuelve al endpoint. La reconstrucción corre en procesos hijos (`ProcessPoolExecutor`) y en funciones del buscador que el agente no instrumenta, así que New Relic no puede ver adentro. La lista de trazas muestra una de 18.917 ms, es decir, 18,9 s: en esa vista New Relic usa el punto como separador de miles.

5.5 El dashboard

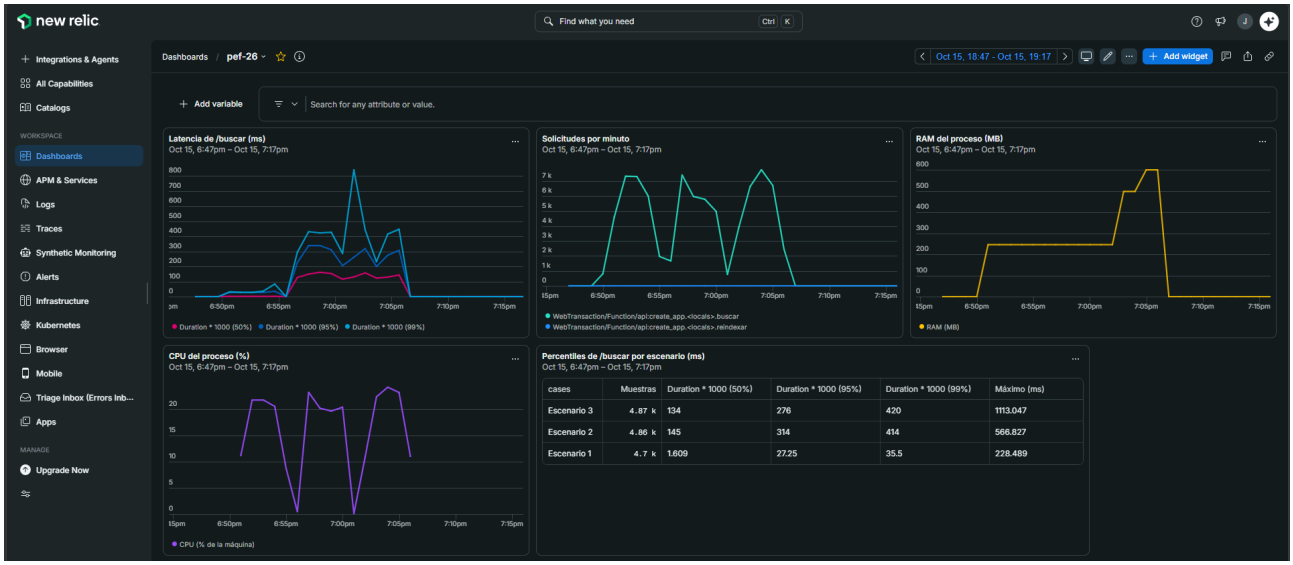


Figura 6. Dashboard pef-26: latencia de cada búsqueda, solicitudes por minuto, RAM y CPU del proceso.

Es el tablero que pide la consigna: cuánto tarda cada búsqueda en tiempo real, el consumo de RAM y las métricas del servidor. Se armó con cinco consultas NRQL (sección 10) sobre la ventana de la campaña:

- **Latencia de /buscar:** percentiles 50, 95 y 99 por minuto. Con un usuario el p99 no llega a 40 ms; con 20 usuarios la mediana ronda los 150 ms y el p99, los 400-500 ms. En el escenario 3 aparecen picos mayores, cuando las recargas compiten por la CPU con las búsquedas.
- **Solicitudes por minuto:** unas 7.000 en los escenarios 1 y 3, y unas 6.400 en el 2. /reindexar aparece como una línea casi en cero.
- **RAM del proceso:** unos 230 MB con el índice cargado, y hasta unos 590 MB por escalones durante las recargas. En macOS el agente informa el pico de memoria del proceso, por eso la línea nunca baja mientras el servidor corre. Después de las 19:07 el servidor está detenido y el gráfico muestra cero.
- **CPU del proceso:** entre el 20 % y el 24 % de la máquina en los tres escenarios; baja en las pausas.

- **Percentiles por escenario:** la tabla de la sección 6.

5.6 Logs

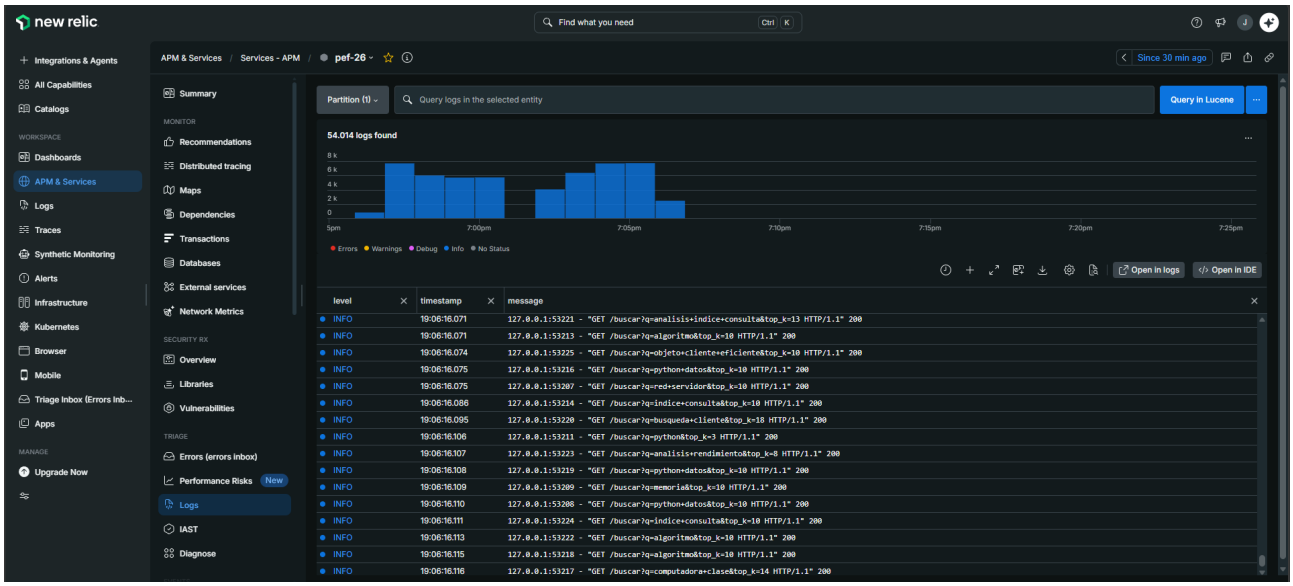


Figura 7. Logs de Uvicorn reenviados por el agente.

El agente reenvía a New Relic los logs de Uvicorn: una línea por solicitud, con la ruta, la consulta y el código de respuesta. La vista abarca los últimos 30 minutos al momento de la captura, que dejan afuera el escenario 1. Muestra 54.014 registros, exactamente las 25.832 + 28.179 búsquedas de los escenarios 2 y 3 más las 3 recargas: New Relic no perdió ningún registro. En el log del servidor, las 81.979 solicitudes de la campaña respondieron 200.

6. Servidor contra cliente

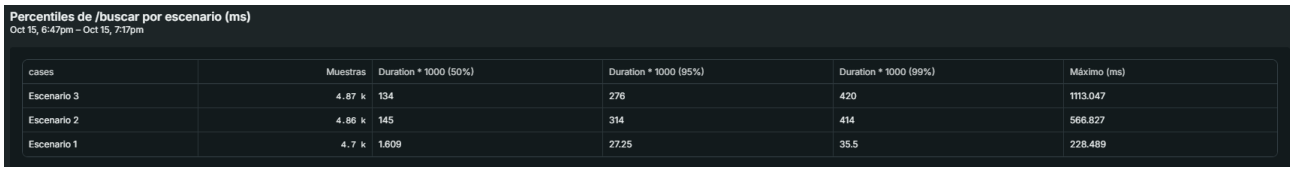


Figura 8. Percentiles de /buscar por escenario, calculados por New Relic.

Escenario	Muestras en New Relic	p50 servidor	p50 cliente	p95 servidor	p95 cliente	p99 servidor	p99 cliente
1	4,7 mil	1,6 ms	3,0 ms	27,3 ms	28,8 ms	35,5 ms	36,3 ms
2	4,86 mil	145 ms	169,4 ms	314 ms	350,9 ms	414 ms	468,6 ms
3	4,87 mil	134 ms	155,6 ms	276 ms	307,9 ms	420 ms	481,9 ms

New Relic mide dentro del servidor, desde que la solicitud llega a la aplicación hasta que sale. El cliente mide además la conexión, la espera en la cola de Uvicorn antes de que la aplicación reciba la solicitud y el envío de la respuesta. Por eso el servidor siempre da menos.

Con un usuario, la diferencia en la mediana es de 1,4 ms: el costo fijo de HTTP. Con 20 usuarios pasa a 22-24 ms en la mediana y a 55-62 ms en el p99: parte de la cola se forma fuera de la aplicación, en el loop de eventos de Uvicorn, que compite por el GIL con los hilos que buscan.

El agente guarda el detalle de hasta 1.200 transacciones por minuto. En 4 minutos son unas 4.800 por escenario, entre el 17 % y el 19 % de las búsquedas. Los percentiles de New Relic son una estimación sobre esa muestra, y aun así quedan cerca de los del cliente. Los promedios y el rendimiento de los gráficos, en cambio, salen de métricas que cuentan todas las solicitudes.

7. Análisis

7.1 El servidor usa un solo núcleo

Con un usuario el servidor atendió 116 búsquedas por segundo; con 20 usuarios, 107 y 117. Multiplicar por 20 los usuarios no aumentó el rendimiento, y la CPU del proceso se quedó en un núcleo de cuatro. Los endpoints son funciones sincrónicas que FastAPI ejecuta en un pool de hilos, pero el ranking es código Python puro y los hilos se turnan el GIL: solo uno avanza a la vez. En mediciones previas de esta etapa, las búsquedas sin caché se estancaron en unas 50 por segundo con cualquier cantidad de usuarios.

7.2 Más usuarios, más espera

Si el rendimiento es fijo, la latencia crece con la cantidad de usuarios. La Ley de Little lo cuantifica: usuarios = rendimiento × tiempo de respuesta. En el escenario 2, 20 / 107,5 por segundo da 186 ms, y el cliente midió una media de 185,8 ms. En el escenario 3, 20 / 117,3 da 170 ms, contra 170,3 ms medidos. Casi todo el tiempo de una búsqueda con 20 usuarios es espera: la consulta repetida, que sale de la caché, tardó 3 ms de mediana con un usuario y 160 ms con 20.

7.3 Las recargas del índice

Una recarga reconstruye con varios procesos el índice de 50.000 documentos y lo guarda en disco. Mientras dura, la API sigue respondiendo con el índice anterior, pero las búsquedas compiten por la CPU: el máximo del escenario 3 llegó a 1,3 s en el cliente (1,1 s en New Relic), contra 0,7 s en el escenario 2. El pico de RAM sube de unos 230 MB a unos 590 MB porque, hasta que termina, conviven el índice viejo y el nuevo.

7.4 Variación entre corridas

La campaña se corrió dos veces el mismo día (la primera quedó con el nombre equivocado). En la primera, el escenario 2 dio 136 búsquedas por segundo; en esta, 107. El servidor y el generador comparten una computadora de 2 núcleos físicos, así que cualquier otro programa activo mueve los números. Las conclusiones valen para las dos corridas: el rendimiento no crece con los usuarios y la latencia sigue la Ley de Little (en la primera, 20 / 136,3 da 147 ms, contra 146,6 ms medidos).

8. Limitaciones

- **Muestreo:** los percentiles de New Relic salen de una muestra del 17-19 % de las búsquedas.
- **RAM:** en macOS el agente informa el pico de memoria del proceso, no la memoria actual, así que la línea solo sube. No incluye los procesos hijos que reconstruyen el índice.
- **CPU:** el dashboard muestra el tiempo de CPU en modo usuario del proceso de la API, como porcentaje de las 4 CPU lógicas. Tampoco incluye los procesos hijos de la recarga.
- **Entorno:** el servidor y el generador corrieron en la misma computadora, sin red de por medio. No representa un servidor de producción.
- **Horarios:** New Relic muestra la hora en UTC, tres horas más que la hora local.

9. Traducción de los indicadores

Indicador original	Traducción	Qué representa
Summary	Resumen	Vista general del servicio.
Web transactions time	Tiempo de las transacciones web	Duración promedio de las solicitudes HTTP.
Throughput (rpm)	Rendimiento (solicitudes por minuto)	Cantidad de solicitudes atendidas.
Error rate	Tasa de errores	Porcentaje de solicitudes con error.
Apdex score	Índice Apdex	Proporción de respuestas satisfactorias según un umbral (T = 0,5 s).
Transactions	Transacciones	Solicitudes agrupadas por endpoint.
Most time consuming	Mayor tiempo consumido	Orden por tiempo total: duración × cantidad.
Breakdown table	Tabla de desglose	En qué funciones se va el tiempo de una transacción.
Transaction traces	Trazas de transacciones	Detalle de solicitudes individuales lentas.
Per-host CPU / memory usage	Uso de CPU / memoria por host	CPU y memoria del proceso de la API en cada equipo.
Median, 95th y 99th percentile	Mediana, percentil 95 y percentil 99	Tiempo por debajo del cual queda el 50, el 95 o el 99 % de las solicitudes.
Logs	Registros	Mensajes del servidor: una línea por solicitud.

10. Cómo reproducir la campaña

Desde la raíz del repositorio, con un archivo `.env` que tenga la línea `NEW_RELIC_LICENSE_KEY=<licencia>`:

```
pip install -r requirements-dev.txt -r requirements-apm.txt
python src/servidor.py --env-file .env

# En otra terminal, con 90 s de pausa entre escenarios:
python src/generar_trafico.py --duracion 240 --concurrencia 1 --semilla 11
python src/generar_trafico.py --duracion 240 --concurrencia 20 --semilla 22
python src/generar_trafico.py --duracion 240 --concurrencia 20 --semilla 33 --reindexar-cada 60
```

Las consultas NRQL del dashboard. Salvo la última, no tienen rango de tiempo: lo fija el selector del dashboard.

Latencia de /buscar (ms)

```
SELECT percentile(duration * 1000, 50, 95, 99) FROM Transaction
WHERE appName = 'pef-26' AND name LIKE '%buscar%' TIMESERIES 1 minute
```

Solicitudes por minuto

```
SELECT rate(count(apm.service.transaction.duration), 1 minute)
FROM Metric WHERE appName = 'pef-26' FACET transactionName
TIMESERIES 1 minute
```

RAM del proceso (MB)

```
SELECT max(apm.service.memory.physical) AS 'RAM (MB)' FROM Metric
WHERE appName = 'pef-26' TIMESERIES 1 minute
```

CPU del proceso (%)

```
SELECT average(apm.service.cpu.usertime.utilization) * 100
AS 'CPU (% de la máquina)' FROM Metric WHERE appName = 'pef-26'
TIMESERIES 1 minute
```

Percentiles de /buscar por escenario (ms)

```
SELECT count(*) AS 'Muestras', percentile(duration * 1000, 50, 95, 99),
  max(duration * 1000) AS 'Máximo (ms)'
FROM Transaction WHERE appName = 'pef-26' AND name LIKE '%buscar%'
FACET CASES (
  WHERE timestamp >= 1792090273000 AND timestamp < 1792090514000
    AS 'Escenario 1',
```

```
  WHERE timestamp >= 1792090604000 AND timestamp < 1792090845000
    AS 'Escenario 2',
  WHERE timestamp >= 1792090935000 AND timestamp < 1792091176000
    AS 'Escenario 3')
SINCE 1792090200000 UNTIL 1792091300000
```

Los números largos son los límites de cada escenario en milisegundos desde 1970 (hora UTC); así la consulta no depende de la zona horaria de quien la ejecuta.

11. Conclusiones

- El requisito queda cubierto: el dashboard de New Relic muestra la latencia de cada búsqueda minuto a minuto, el consumo de RAM y la CPU del proceso, junto con el rendimiento y los errores.
- La API es estable bajo carga: 81.979 solicitudes sin errores, también durante las recargas del índice.
- El cuello de botella es la CPU de un solo núcleo: el rendimiento tiene un techo y la latencia crece con los usuarios. Para mejorar se puede atender con varios procesos de Uvicorn, cada uno con su propia copia del índice (unos 230 MB), o reducir el costo de cada búsqueda sin caché.
- New Relic no ve dentro de las funciones del buscador. El perfilado del punto 8, en la sección «Perfilado de la versión refactorizada» del README, muestra qué funciones y líneas consumen ese tiempo.

12. Guion breve para la exposición

“Instrumenté la API con el agente de New Relic, sin tocar el código del buscador, y le apliqué carga con un generador propio en tres escenarios: un usuario, veinte usuarios y veinte usuarios con recargas del índice. El dashboard muestra cuánto tarda cada búsqueda minuto a minuto, la RAM y la CPU. Con un usuario, la mediana de una búsqueda en el servidor es de 1,6 ms; con veinte, de 145 ms, pero el servidor atiende la misma cantidad por segundo, porque la CPU se queda en un núcleo: el GIL de Python hace que los hilos se turnen. Las recargas tardan 16 segundos y llevan el pico de RAM de 230 a 590 MB, sin errores. New Relic me dice qué es lento; para saber por qué, uso cProfile con SnakeViz y Scalene.”